

5.5 Itens: foto do item — status de entrega

Bloco 5.5 Itens: foto do item · Sprint 5 — Itens · Entregue em 2026-09-22, ajustado em 2026-09-23 · Branch: feature-dev-breno-automacao · Ambiente: gc-dev (gzbmhgnpoehormnidmgg)

Resumo

Entregue só o **CRUD da imagem no servidor**, por decisão do Breno em 2026-09-22: sem tela, sem miniatura, sem diálogo, e sem teste automatizado que crie ou suba imagem. O teste com imagem de verdade é manual e fica com o Breno (Seção 4).

Uma interface completa chegou a ser construída e foi removida inteira antes da entrega: o `foto-item-dialog.tsx` foi apagado, a coluna de miniatura saiu da tabela e o `FileUpload` voltou ao original.

1. As quatro atividades do bloco

ATIVIDADE DO CLICKUP	ESTADO
Reusar <code>FileUpload</code> com bucket <code>anexos</code> e path <code><empresa_id>/itens/<item_id>/</code>	bucket e path sim ; <code>FileUpload</code> não , porque não há tela
Gravar em <code>itens.foto_url</code>	sim — o path, não a URL (2.1)
Miniatura na tabela e visualização ampliada ao clicar	fora do escopo por decisão
Substituir e remover foto	sim , no servidor

2. O que foi entregue

Três Server Actions em `src/app/(app)/propostas/[id]/itens-actions.ts` e um helper puro, `src/lib/fotos.ts` (9 exports, 8 testes).

ACTION	O QUE FAZ
<code>uploadFotoItem(propostaId, itemId, formData)</code>	sobe a foto e grava o path; se já havia foto, substitui
<code>removeFotoItem(propostaId, itemId)</code>	tira do Storage e zera <code>foto_url</code>
<code>deleteItem</code> / <code>deleteProposta</code>	passam a remover a foto junto com o item

2.1 `foto_url` guarda o path, não a URL

O nome da coluna sugere URL, mas o bucket `anexos` é **privado**. URL assinada expira em uma hora, então gravá-la deixaria todas as fotos quebradas no dia seguinte. O valor gravado é `<empresa_id>/itens/<item_id>/<timestamp>_<nome>`, e a URL se gera na hora de mostrar — o mesmo

desenho que os anexos de proposta usam (`Anexo.path`). Quando houver tela, ela chama `createSignedUrls` em lote.

2.2 A ordem da substituição, e por que ela importa

1. sobe o arquivo **novo** num path novo (o timestamp no nome impede colisão);
2. troca o ponteiro `foto_url` ;
3. remove o **antigo**.

Se o passo 2 falha, o arquivo novo é removido e nada mudou. Se o passo 3 é negado, o ponteiro volta ao antigo e o novo é removido: **a substituição é recusada inteira**, em vez de deixar um arquivo órfão que ninguém vê e ninguém consegue apagar.

2.3 A regra de dono do Storage

A policy de delete do bucket `anexos` é **admin ou quem enviou o arquivo** (`owner = auth.uid()`). Por isso um comercial não substitui nem remove a foto que outro comercial (ou um admin) enviou, e recebe "Só quem enviou a foto atual ou um admin pode substituí-la".

Um detalhe que custou atenção: o **`remove()` do Storage não devolve erro quando o RLS nega** — devolve sucesso com a lista vazia. `removeuTudo()` transforma "pedi 1, removeu 0" no que isso é: falha. Sem ela, a action responderia `ok` e o arquivo continuaria lá.

2.4 O que é aceito

JPG, PNG e WebP, até 10 MB. É a interseção entre o que o bucket aceita e o que é foto:

- **GIF fica de fora** — o bucket não aceita. Na entrega original `src/lib/files.ts` o aceitava para anexos de proposta, que falhavam no Storage; corrigido no fechamento.
- **HEIC fica de fora** — é o formato nativo do iPhone, aceito só no bucket `evidencias` .

2.5 `pathEhDoItem` antes de apagar qualquer arquivo

Nenhuma action remove um arquivo sem conferir que o path está dentro de `<empresa>/itens/<item>/` , sem `..` . O path vem do banco, mas `foto_url` já foi gravável por qualquer POST até a correção abaixo, e remover do Storage o arquivo de outro item seria destruir dado alheio.

3. Uma correção de segurança encontrada aqui (código do 5.2)

Preparando o 5.5 apareceu um furo nas actions de item do 5.2. Elas montavam o registro com `{ empresa_id, obra_id, proposta_id, ..., ...input }` — o corpo da requisição vinha **por último** e sobrescrevia o que o servidor tinha resolvido. `ItemFormInput` não tem `proposta_id` , mas tipo não existe em runtime: um POST podia mover o item para outra proposta (furando a guarda de rascunho), trocar `created_by` , ou apontar `foto_url` para o arquivo de outro item. O RLS barra outra empresa; dentro da mesma, nada barrava.

Correção: `CAMPOS_EDITAVEIS_ITEM` e `camposEditaveisItem()` em `src/lib/itens.ts` — uma whitelist de 13 campos, e os do servidor aplicados **depois**. `foto_url` ficou de fora dela: só as actions de foto a escrevem.

Junto veio outro: a **exclusão de item aceitava comercial**, mas a policy "Itens: admin exclui" só deixa admin. O RLS negava em silêncio (delete negado devolve 0 linhas, não erro) e a action respondia `ok` — a tela dizia "Item excluído" e o item continuava lá. Agora o guard exige admin, a lixeira só aparece para admin, e a action confere as linhas apagadas.

Os dois estão registrados também no documento do 5.2.

4. Números reais do `npm run validar`

Rodado em 2026-09-22 contra gc-dev. Os números valem para os blocos 5.5 e 5.6 juntos, porque foram entregues na mesma rodada.

CAMADA	ANTES (FIM DO 5.4)	AGORA	
estático	limpo	<code>tsc --noEmit</code> limpo, <code>✓ No ESLint warnings</code> <code>or errors</code>	ok
unitário	106 casos	124 casos, 0 falhas	+8 deste bloco, +10 do resto
build	32 rotas	32 rotas, iguais à baseline	=
runtime	44/44 rotas	47/47 rotas ok	+3 (5.6)
dados	15/15 checagens	18/18 checagens ok	+3 (5.6)
escrita	73/73 passos	90/90 passos ok	+3 (correção do 5.2), +14 (5.6)
navegador	41/41 passos	41/41 passos ok , console limpo	=

Deste bloco especificamente:

- **8 casos unitários** em `src/lib/fotos.test.ts`: tipos aceitos e recusados (GIF, HEIC, PDF), vazio e acima de 10 MB, que os tipos são subconjunto do bucket, o prefixo começando pela empresa, `pathEhDoItem` recusando outro item, outra empresa, pasta sem arquivo e `..`, e `removeuTudo` tratando lista vazia como falha. **Nenhum cria arquivo**.
- **3 passos de escrita** da correção de segurança: `createItem` ignora `proposta_id`, `created_by` e `foto_url` do corpo; `updateItem` não move o item de proposta nem aceita `foto_url`; comercial não exclui item e a resposta diz isso, não "ok".

Estado do banco depois da rodada, conferido por query: **0** propostas de teste, **0** itens com foto, **0** arquivos de item no bucket, **0** itens órfãos.

Fechamento da sprint 5 — o que mudou neste bloco

Revisão de 2026-09-23: os seis documentos da sprint foram relidos, e cada pendência nominal foi corrigida ou justificada. Números do plano inteiro no fim da revisão:

CAMADA	BASELINE (SPRINT 4)	FIM DA SPRINT 5
estático	limpo	<code>tsc --noEmit</code> limpo, <code>✓ No ESLint warnings or errors</code>
unitário	51 casos	140 casos, 0 falhas
build	31 rotas	32 rotas, iguais à baseline
runtime	41/41 rotas	47/47 rotas ok
dados	12/12 checagens	18/18 checagens ok
escrita	38/38 passos	101/101 passos ok
navegador	24/24 passos	50/50 passos ok, console limpo

Rodado em 2026-09-23 contra gc-dev. `migration list` : 17/17 alinhadas. Depois da rodada, conferido por query: 0 propostas de teste, 0 contratos de teste, 0 itens órfãos, 0 arquivos de item no bucket.

Resolvido:

- Anexo de proposta em GIF (pendência 4).** Confirmado e corrigido. `src/lib/files.ts` aceitava GIF, que o bucket `anexos` recusa — o anexo passava na tela e falhava no upload com erro cru do Storage — e não aceitava WebP, que o bucket aceita. Agora o cliente aceita exatamente o que o bucket aceita, e `src/lib/files.test.ts` lê a lista do bucket direto da migration e compara: se alguém mudar um lado e esquecer o outro, o teste quebra. O texto da área de upload passou a dizer WebP em vez de GIF.

As três actions deste bloco (`uploadFotoItem`, `removerFotoItem` e a remoção junto com o item e com a proposta) não mudaram na revisão.

5. O que NÃO foi validado — pendências nominais

- Nenhuma imagem foi enviada por teste automatizado, por decisão: é o teste manual do Breno.**
Roteiro:
 - admin envia foto → `foto_url` preenchido com path em `<empresa>/itens/<item>/` ;
 - admin envia outra → path muda e o arquivo antigo some do bucket;
 - comercial tenta substituir ou remover a foto do admin → recusa com mensagem, e nada muda no banco nem no bucket;
 - admin remove → `foto_url` nulo e bucket vazio para o item;
 - admin exclui o item com foto → a foto sai do bucket;
 - enviar GIF ou arquivo acima de 10 MB → recusa com mensagem.

2. **Não há como enviar foto pela tela**, por decisão: o bloco entrega só o servidor.
3. **HEIC não é aceito**. O bucket `anexos` não aceita, e aceitar mudaria o bucket. Foto de iPhone costuma chegar como JPEG pelo Safari, mas não é garantido — vale observar no teste manual.

6. Próximo passo

Quando houver tela para foto, ela usa estas actions como estão, e gera as URLs com `createSignedUrls` em lote a partir de `foto_url`.